C++20 and beyond: improving embedded systems performance

Alfredo Muela

using std::cpp 2026

The conversation that goes nowhere

"It will be slower with all those extra"

"We don't have time for experiments"

"Prove it won't be suboptimal"

The problem isn't that these fears are unreasonable

The problem is that we answer them with opinions instead of measurements

What if you could show them?

Four categories of objective evidence, each with a tool

EVIDENCE	TOOL
Binary impact & data placement	**arm-none-eabi-size (sections)**
Code generation quality	**Godbolt / arm-none-eabi-objdump**
Code complexity	**lizard / clang-tidy metrics**
Compiler diagnostics	**direct comparison of error output**

Synthetic but realistic — Cortex-M4 target with -Os -flto -fno-exceptions

The code compiles. The measurements are real. Repo linked at the end.

Concepts + constexpr + static_assert

I²C driver interface — the C++17 version

// C++17: SFINAE trait detection (i2c_traits.hpp)

```cpp
#include <type_traits>
#include <cstdint>

// --- Trait: has read_register(uint8_t) -> uint16_t ---
template<typename T, typename = void>
struct has_read_register : std::false_type {};

template<typename T>
struct has_read_register<T, std::void_t<
    std::enable_if_t<std::is_same_v<
        decltype(std::declval<const T&>().read_register(uint8_t{})),
        uint16_t
    >>
>> : std::true_type {};

template<typename T>
inline constexpr bool has_read_register_v = has_read_register<T>::value;

```

// ~45 lines of boilerplate

Compiler error when passing wrong type

```
error: no type named 'type' in 'struct std::enable_if<false, void>'
  In substitution of 'template<class T, class> class I2CDriverCpp17...'
  ... 30+ lines of nested template instantiation context ...
  ... actual requirement buried at line 34 ...
```

Concepts + constexpr + static_assert

I²C driver interface — the C++20 version

// C++20: concept definition (i2c_concepts.hpp)

```cpp
#include <concepts>
#include <cstdint>

template<typename T>
concept I2CSensor = requires(T dev, const T cdev) {
    { cdev.read_register(uint8_t{}) } -> std::same_as<uint16_t>;
    { dev.write_register(uint8_t{}, uint16_t{}) } -> std::same_as<bool>;
    { T::device_address } -> std::convertible_to<uint8_t>;
};
```

// That's all folks

Same mistake, same compiler

```
error: constraints not satisfied for 'I2CDriverCpp20<WrongType>'
note: 'WrongType' does not satisfy 'I2CSensor'
note: 'read_register' not found
```

34 lines -> 3 lines

What the tools say

METRIC	TOOL	BEFORE (C++17)	AFTER (C++20)
.text	arm-none-eabi-size	(identical)	(identical)
Diagnostic clarity	compiler output	34 lines	3 lines
Config errors caught	static_assert	0 (runtime crash)	compile time

Zero binary cost. 10× clearer diagnostics

A class of errors that physically cannot reach production

Bring these numbers from YOUR project to your stakeholder meeting

+10% compilation time

The sensor data pipeline

Filter, calibrate, and average sensor readings

C++17

```
// 3 loops, 2 intermediate buffers
std::array<float, MAX_READINGS> filtered{};
std::size_t fcount = 0;
for (std::size_t i = 0; i < raw_count; ++i) {
    if (in_range(raw[i])) {
        if (fcount >= MAX_READINGS) break;
        filtered[fcount++] = raw[i];
    }
}

std::array<float, MAX_READINGS> calibrated{};
for (std::size_t i = 0; i < fcount; ++i) {
    calibrated[i] = calibrate(filtered[i]);
}

// ... third loop for sliding window average ...
```

C++20 + ranges-v3

```
// Lazy filter + calibrate pipeline
for (float v : raw
    | views::filter(in_range)
    | views::transform(calibrate)) {
    if (cal_count >= MAX_READINGS) break;
    calibrated[cal_count++] = v;
}

// Sliding window (forward range required)
auto windowed =
    std::span(calibrated.data(), cal_count)
    | views::sliding(SLIDING_WINDOW_SIZE)
    | views::transform(window_avg);
```

No extra buffer. No stale data. Intent readable left-to-right

What Godbolt says

ARM Thumb2 assembly — hot loop, -Os -DNDEBUG -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16 -fno-exceptions -fno-rtti

loops C++17

```
.L13:
    vldmia.32  r6!, {s13}
    subs    r0, r0, #1
    vadd.f32   s15, s15, s13
    bne     .L13

.L12:
    vldr.32 s15, .L22+12  ; sum=0
    mov     r6, r3        ; win start
    movs    r0, #8        ; counter

    ; --- inner (.L13) ---

    vmul.f32  s15, s15, s14  ; avg
    adds    r1, r1, #1       ; count++
    cmp     r4, r1           ; windows left?
    vstmia.32 r2!, {s15}     ; store
    bcc     .L15             ; done
    cmp     r5, r1           ; capacity?
    add     r3, r3, #4      ; advance
    bne     .L12
```

ranges

```
.L27:
    vldmia.32  r3!, {s13}
    vadd.f32   s15, s15, s13
.L29:
    cmp     r3, r5
    bne     .L27

.L26:
    cmp     r1, r4        ; done?
    beq     .L17          ; return
    vldr.32 s15, .L36+8  ; sum=0
    mov     r3, r1        ; win start
    add     r5, r1, #32  ; win end
    b       .L29
    ; --- inner (.L27/.L29) ---
    vmul.f32  s15, s15, s14  ; avg
    cmp     r0, r6           ; capacity?
    beq     .L17             ; return
    adds    r0, r0, #1       ; count++
    vstmia.32 r2!, {s15}     ; store
    adds    r1, r1, #4       ; advance
    b       .L26
```

Same structure, register allocation differs, but the operations are instruction-for-instruction **equivalent**

tl::expected – feat. full monadic surface

Error handling + recovery + fallback — 4 steps

C++17

```
SensorState init_sensor(hw::I2CDevice& dev) {
    auto err = configure(dev);
    if (is_ok(err)) {
        err = read_calibration(dev);
        if (is_ok(err)) {
            err = read_offset(dev);
            if (is_ok(err)) {
                err = validate(dev);
                if (is_ok(err)) {
                    return SensorState{get_config(dev),
                                       Mode::Sampling};
                }
            }
        }
    }
    log_init_failure(dev, err);
    auto fb = safe_mode_config(dev);
    if (is_ok(fb)) {
        return SensorState{get_config(dev), Mode::Degraded};
    }
    return SensorState::offline();
}
```

C++20 + tl::expected & monadic

```
SensorState init_sensor(hw::I2CDevice& dev) {
    return configure(dev)
        .and_then(read_calibration)
        .and_then(read_offset)
        .and_then(validate)
        .transform([](DevRef ref) {
            return SensorState{get_config(ref.get()),
                               Mode::Sampling};
        })
        .or_else([&](ErrorCode err)
            -> tl::expected<SensorState, ErrorCode> {
            log_init_failure(dev, err);
            return safe_mode_config(dev)
                .transform([](DevRef ref) {
                    return SensorState{get_config(ref.get()),
                                       Mode::Degraded};
                });
        })
        .value_or(SensorState::offline());
}
```

Same logic. Success path buried in nesting, error path separated at the bottom vs success and error paths compose in a single expression

tl::expected – feat. full monadic surface

Maintenance time! What changes when you add a 5th step to each version?

C++17

```
SensorState init_sensor(hw::I2CDevice& dev) {
    auto err = configure(dev);
    if (is_ok(err)) {
        err = read_calibration(dev);
        if (is_ok(err)) {
            err = read_offset(dev);
            if (is_ok(err)) {
                err = compensate_temp(dev);
                if (is_ok(err)) {
                    err = validate(dev);
                    if (is_ok(err)) {
                        return SensorState{get_config(dev),
                                           Mode::Sampling};
                    }
                }
            }
        }
    }
    log_init_failure(dev, err);
    auto fb = safe_mode_config(dev);
    if (is_ok(fb)) {
        return SensorState{get_config(dev), Mode::Degraded};
    }
    return SensorState::offline();
}
```

C++20 + tl::expected & monadic

```
SensorState init_sensor(hw::I2CDevice& dev) {
    return configure(dev)
        .and_then(read_calibration)
        .and_then(read_offset)
        .and_then(compensate_temp)
        .and_then(validate)
        .transform([](DevRef ref) {
            return SensorState{get_config(ref.get()),
                               Mode::Sampling};
        })
        .or_else([&](ErrorCode err)
            -> tl::expected<SensorState, ErrorCode> {
            log_init_failure(dev, err);
            return safe_mode_config(dev)
                .transform([](DevRef ref) {
                    return SensorState{get_config(ref.get()),
                                       Mode::Degraded};
                });
        })
        .value_or(SensorState::offline());
}
```

What the tools say

METRIC	TOOL	BEFORE (C++17)	AFTER (C++20)
Cyclomatic complexity	lizard	CCN 6	CCN 1
Branch structure	Godbolt	N conditional jumps	N conditional jumps
Adding a new step	git diff	+3 lines, 5 re-indent	+1 line

Same binary. Same branches. Lower complexity. **One-line extensibility**

The compiler doesn't care — but your code reviewer does, and your future self-maintaining this at 2 AM certainly does

tl::expected → std::expected (C++23). Migration = namespace change

consteval + bit_cast + constinit + span + GSL

Compile-time calibration tables

C++17 (runtime init → .bss)

```
// calibration_table.cpp
std::array<float, ADC_TABLE_SIZE> cal_table{};

void init_calibration(const SteinhartCoeffs& coeffs)
{
    for (int i = 0; i < ADC_TABLE_SIZE; ++i) {
        float T = steinhart_hart(adc_to_resistance(i), coeffs);
        uint32_t bits;
        std::memcpy(&bits, &T, sizeof(bits));
        // runtime NaN check
        if ((bits & 0x7F800000u) == 0x7F800000u)
            T = CALIBRATION_FALLBACK_TEMP;
        cal_table[static_cast<std::size_t>(i)] = T;
    }
}
```

C++20 (consteval → .rodata / .data)

```
// calibration_table.hpp
consteval auto gen_cal_table(SteinhartCoeffs c)
    -> std::array<float, ADC_TABLE_SIZE>
{
    std::array<float, ADC_TABLE_SIZE> t{};
    for (int i = 0; i < ADC_TABLE_SIZE; ++i) {
        float T = steinhart_hart(adc_to_resistance(i), c);
        auto b = std::bit_cast<uint32_t>(T);
        // compile-time NaN detection
        if ((b & 0x7F800000u) == 0x7F800000u)
            throw "NaN in calibration";
        t[static_cast<std::size_t>(i)] = T;
    }
    return t;
}

constinit std::array<float, ADC_TABLE_SIZE> runtime_cal_table =
    gen_cal_table(default_coeffs);
```

Storage: `constexpr` → `.rodata` (flash) `constinit` → `.data` (RAM, field-recalibratable)
Consumer: `std::span<const float>` (decoupled from storage, carries bounds)
Safety: `gsl::narrow<uint8_t>(adc >> 4)` (checked index truncation)

What the tools say

arm-none-eabi-size reveals WHERE data lives, not just how much

C++17 init_calibration

```
init_calibration:
    push {r4-r7, lr} ; 30+ instrs
    ; loop, FP math, memcpy, NaN check
    ; ... runs once at startup ...
    pop {r4-r7, pc}
```

C++ consteval

SECTION	C++17	C++20	DELTA
.text (code)	1732 B	1168 B	-564 B
.data (init RAM)	84 B	4 B	-80 B
.bss (zero-init RAM)	1348 B	8 B	-1340 B
total	3164 B	1180 B	-1984 B

Considerations on the section map

`constexpr → .rodata (flash). Immutable. Zero RAM cost`

For sensors with fixed calibration curves, CRC polynomials, pin maps

`constinit → .data (initialized RAM). Mutable. No startup code`

For sensors requiring field recalibration. No SIOF. No init function

`bit_cast → enables compile-time validation. Same syntax at runtime`

Without it, consteval + float inspection is impossible (memcpy is not constexpr)

If already using `arm-none-eabi-size` to check total size, why not start asking: which section? flash or RAM? Mutable or const?

That question may change how you think about every data structure in your firmware

Caveat: consteval tables work for compile-time-known data. For field-calibrated data, constinit gives you compile-time init with runtime mutability. The two complement each other.

The toolkit you take home

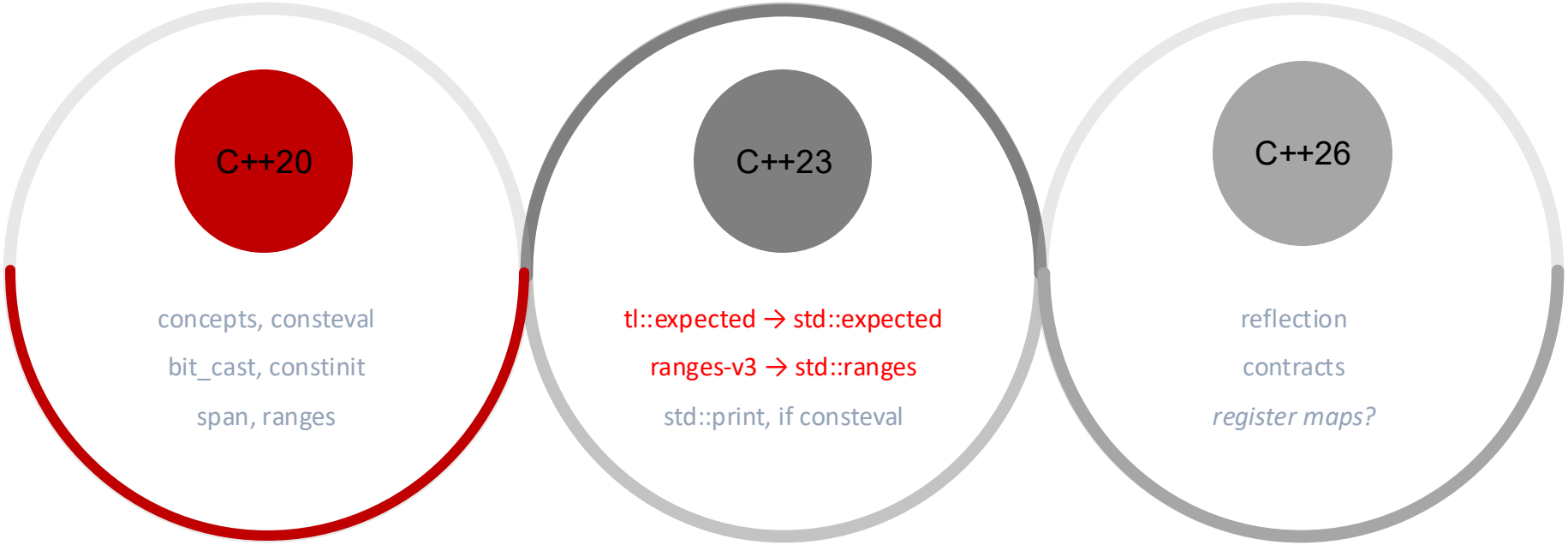
FEATURE	TOOL	WHAT TO LOOK FOR
concepts + constexpr	compiler diagnostics	binary neutral? diagnostics shorter? configs validated at compile time?
ranges v3 pipelines	arm-none-eabi-size, Godbolt	.text modest increase? lazy? assembly equivalent?
tl::expected chains	lizard (complexity), Godbolt, git diff	complexity drop? same branches? one-line extensibility?
consteval + constinit + bit_cast + span	arm-none-eabi-size (section analysis), objdump	.text smaller (init gone)? .bss → .rodata/.data? validation at compile time?

Run these on **your** code, with **your** toolchain, on **your** target

The evidence you need for your case is what your compiler produces, not what I tell you to believe

And the human factors (measure also in your context): migration effort (days), review friction (ask your team), defect classes eliminated (issue tracker), extensibility (next feature), development happiness (attrition/engagement)

The bridge forward



Supplement libraries are not a detour, they're a **bridge**

Thank you

Goldbolt's

https://using26-demo1.godbolt.org/z/er9nvn3Eh

https://using26-demo2.godbolt.org/z/4Pzjq74zv

https://using26-demo4.godbolt.org/z/Pvbsqoshx

GitHub

https://github.com/amuela-verisure/talk-usingstd-2026

Don't take my word. Take your compiler's

Contact

alfredo.muela@verisure.com

verisure